ZVDetector: State-Guided Vulnerability Detection System for Zigbee Devices

Hai Lin
Tsinghua University
Beijing, China
linhai22@mails.tsinghua.edu.cn

Chenglong Li*
Tsinghua University
Beijing, China
lichenglong@tsinghua.edu.cn

Jiahai Yang*
Tsinghua University
Beijing, China
yang@cernet.edu.cn

Zhiliang Wang
Tsinghua University
Beijing, China
wzl@cernet.edu.cn

Jiaqi Bai
Tsinghua University
Beijing, China
bjq21@mails.tsinghua.edu.cn

Abstract

Nowadays, Zigbee devices are widely used in smart home, smart agriculture and other industries. However, there are many vulnerabilities in Zigbee devices that could compromise their normal functionality. Existing research either analyzes firmware or fuzzes devices through Zigbee networks to discover potential vulnerabilities. However, they overlook the impact of device state and protocol state on firmware or explore only a limited state space. Thus, they fail to identify many vulnerabilities caused by hidden states within each of the two states, especially vulnerabilities triggered by the combination of these two states. In this paper, we design a state-guided fuzzing system, named ZVDetector, aimed at uncovering firmware vulnerabilities caused by hidden and combined states. Specifically, we design two state-aware modules that explore richer unknown protocol state transitions based on message relationships and gain a more complete understanding of the intrinsic device state attributes. We develop a fuzzing algorithm that incorporates message semantics awareness and correlation state analysis. By integrating the perceived state information, it can explore the combined state space more efficiently. We validate the performance of ZVDetector on 10 Zigbee devices and find 25 vulnerabilities (19 zero-day). Our experiments also demonstrate the ability to explore more device state attributes and discover more message relationships related to unknown protocol states.

CCS Concepts

• **Security and privacy → Mobile and wireless security**.

Keywords

Vulnerability Detection, Device State, Protocol State, Fuzzing

ACM Reference Format:
Hai Lin, Chenglong Li, Jiahai Yang, Zhiliang Wang, and Jiaqi Bai. 2025. ZVDetector: State-Guided Vulnerability Detection System for Zigbee Devices. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25), October 13–17, 2025, Taipei.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3719027.3765035

*Chenglong Li and Jiahai Yang are corresponding authors.

CCS '25, Taipei

ACM ISBN 979-8-4007-1525-9/2025/10
https://doi.org/10.1145/3719027.3765035

1 Introduction

With the rapid development of computer technologies such as embedded systems and wireless communications, IoT (Internet of Things) devices have been widely deployed all around the world. A report [14] shows that the Zigbee market size has reached $4.87 billion as of 2024 and is expected to reach $6.51 billion by 2029. However, security issues about Zigbee devices are discussed widely [5, 10, 11]. One type of attack exploits protocol vulnerabilities during the pairing phase. Attackers can exploit the Zigbee touchlink vulnerability to launch a denial-of-service (DoS) attack from one device, causing the target device to crash [37], or they can eavesdrop to obtain encryption keys [26, 42, 51]. By spoofing addresses and identity information contained in the plaintext messages transmitted [44, 46, 48], attackers can gain control over the device. Another type of security issue occurs during the communication phase, usually caused by firmware vulnerabilities, including null pointer dereference [20, 27], buffer overflow [20, 40], and DoS [1–3, 27].

To detect firmware vulnerabilities, some studies have employed static analysis methods [18, 19, 38, 43] to analyze firmware, such as symbolic execution [24, 32] and taint analysis [22]. However, the firmware of many Zigbee devices is not open source, and the programming languages used by different manufacturers vary significantly. Therefore, these methods are challenging to generalize. Similarly, dynamic emulation [19, 33, 41, 49, 50] also face difficulties in generalization due to differences in instruction set architectures and the dynamic environment in which the firmware runs.

Additionally, many studies utilize companion application of IoT devices and apply fuzzing on the message construction path to discover firmware vulnerabilities. Snipuzz [27] employs a black-box fuzzing approach to identify firmware vulnerabilities. It collects initial network messages through API-testing programs and mutates these messages before sending them to the device. IoTFuzzer [20] and Diane [40] are two similar grey-box fuzzers that analyze the companion app code of IoT devices to send mutated messages that cover critical paths. However, due to the open-source issues of testing programs and the heterogeneity of application code, these methods face challenges in achieving general applicability.

Besides, much research [21, 23, 25, 28, 31, 39, 45] uses protocol state fuzzing for protocol vulnerability detection, but they are designed for other specific protocols, such as TLS [25, 28] and VPN [23]. These approaches learn the message dependency by analyzing RFC documents or running protocols and then construct mutated messages to break these dependencies. However, there are more complex relationships between Zigbee messages and scenarios in which the relationships exist. For example, the message correlations may exist in the cross-device interaction scenario. Therefore, these approaches fail to explore the protocol state sufficiently.

To address this problem, fuzzing for protocol message structures [35, 36, 41, 47, 48] has less impediments for generalization and can be combined with state fuzzing to achieve the detection of protocol vulnerabilities and firmware vulnerabilities. Ren et.al [41], Wang et al. [48], Meng et al. [36] and LLMIF [47] obtain the format of various protocol messages, which are used to guide the subsequent format-aware fuzzing tests to generate effective test cases. HubFuzzer [35] further analyzes the supported functions of the device and mutated messages based on these functions. These methods are effective in discovering vulnerabilities, but have the following three limitations. **(1)** They ignore the impact of the **protocol state** on the message reception or explore a limited protocol state space. Under certain protocol state, receiving a specific message or a message sequence will potentially trigger vulnerabilities. Most studies focus only on single-message fuzzing. Existing protocol state fuzzing approaches and LLMIF generate mutated message sequences based on message relationships. However, they either only collect message dependencies or just analyze read-write correlations. **(2)** They ignore the impact of the internal **device state** on message reception or explore a limited device state space. Receiving messages in a specific device state can trigger device vulnerabilities. However, Zigbee devices contain complex attributes internally with various permissions, such as the presence of hidden states that are not explicitly defined and ReadOnly attributes. HubFuzzer only considers specification-defined attributes with write permissions. **(3)** Most of them ignore the exploration of the **product space** of device states and protocol states. In a specific device state, receiving a mutated message sequence that may trigger potential protocol state changes could lead to firmware vulnerabilities. HubFuzzer only considers fuzz single protocol message under corner device state setting.

Our Approach. In this paper, we introduce ZVDetector, a vulnerability detection system for Zigbee devices that efficiently explores the protocol. Given the large number of Zigbee messages, we adopt a message relationship-based protocol state exploration method instead of exhaustively testing all message combinations. To address **limitation (1)**, we combine static analysis (including LLM techniques) and dynamic analysis to completely capture message dependencies and correlations. We then propose three heuristic relationship-based strategies to explore unknown protocol state transitions triggered by certain messages within the constructed protocol state graph and to generate the fuzzing graph. There are complex internal state attributes and access permissions within the device, which leads to **limitation (2)**. To tackle this, we develop a multi-source data mining approach for collecting state attributes, incorporating active probing, specification documents and development document analysis, and LLM-assisted message analysis. We observe that many ReadOnly attributes can be modified via certain commands and acquire a fine-grained understanding of attribute permissions with the help of LLM. Finally, to address the inefficiency of black-box fuzzing and the insufficient exploration of the combined state space described in **limitation (3)**, we design a state-guided fuzzing algorithm. The state-guided part generates test messages based on the fuzzing graph and explores only the device states related to the messages, rather than the entire state space. The mutation part adopts a message semantics-aware mutation strategy to reduce test cases being sanitized. Additionally, we adjust new test cases based on crash feedback, focusing on combined states that trigger vulnerabilities.

Contribution. We make the following contributions:

- We introduce a novel system, called ZVDetector, which can identify Zigbee device vulnerabilities triggered by the combination of protocol states and device states. Our combined state fuzzing methodology can also be applied to other IoT protocols.[1]
- We propose a method for exploring unknown protocol states efficiently and sufficiently. Various message relationships are mined through the combination of static and dynamic analysis and leveraged with a heuristic relationship strategy to identify messages that trigger unknown protocol states in the state graph.
- We develop a technique for fine-grained awareness of device states. Internal device state attributes are obtained through a three-stage multi-source data analysis and combined with an LLM-based analysis method to determine attribute permissions, enabling the fuzzer to explore more device states.
- We design a state-guided fuzzing algorithm that efficiently explores the combined state space. The message semantics-aware mutation strategy reduces invalid test cases, while correlated device state analysis and crash-based feedback adjustment enable faster identification of device vulnerabilities.
- We evaluate our approach on ten devices with 5 vendors. We have discovered 25 vulnerabilities among 7 devices, 7 CVE IDs are assigned and 19 of them are zero-day.

2 Background

In this section, we first introduce the Zigbee protocol stack and Zigbee devices. Then we describe the motivation, threat model and scope, and the challenges associated with each motivation. Here, we also briefly outline the solutions for each challenge, which align with the components of the system framework described in **§3**.

2.1 Zigbee Stack & Zigbee Device

Zigbee is a low-power, lightweight IoT communication protocol that is widely used in smart homes and medical information collection. The Zigbee protocol stack has five layers from top to bottom, namely Zigbee Cluster Library (ZCL) layer, Application Support Sublayer (APS), Network (NWK) layer, Media Access Control (MAC) layer and Physical layer. The MAC layer and NWK layer are used to identify the Zigbee network and device routing, while the APS layer and ZCL layer contain messages for controlling Zigbee devices.

[1] **Open source code at:** *https://github.com/ZVDetector/ZVDetector/tree/master*

Table 1: Comparison of the state-of-the-art (SOTA) vulnerability detection method with ZVDetector, where ○ means no support, ◐ means partially support, while ● means totally support. [S] and [M] refer to fuzzing of single protocol message and fuzzing of the sequence of multiple messages respectively, [B] and [H] refer to basic attribute collection and hidden attribute collection respectively.

Method \ Support	Protocol State Awareness		Device State Awareness		Combination State Fuzzing
	Message Relationship Analysis	Protocol State Fuzzing [S] [M]	State Attribute Collection [B] [H]	State Attribute Permission Analysis	
Meng[36], DIANE[40], Z-Fuzzer[41], Wang[48]	○	● ○	○ ○	○	○
HubFuzzer[35]	○	● ○	◐ ○	◐	◐
PULSAR[31], Chen[21]	○	● ◐	○ ○	○	○
TLS, DTLS[25, 28, 30]	◐	● ◐	○ ○	○	○
LLMIF[47]	◐	● ◐	○ ○	○	○
ZVDetector	●	● ●	● ●	●	●

Zigbee Device Profile (ZDP) belongs to APS layer and is used for device discovery and management.

Coordinators (also called controllers) are considered to be the central nodes responsible for the addresses distribution when devices join, as well as for data packets routing. Each coordinator creates a localized network identified by a 16-bit Personal Area Network Identifier (PAN ID) and a 64-bit Extend PANID (EPID). Each Zigbee device (also called end device) has a 16-bit identified MAC address and a 16-bit network address assigned by the coordinator.

Each Zigbee device has multiple endpoints. An endpoint is a logical entity that provides a service and identified by an 8-bit integer. ZCL **clusters** are contained within each endpoint and implement some specific functions, where each cluster is represented by a 16-bit identifier called cluster ID (CID). Each cluster provides a set of related **attributes** and **commands**, e.g. cluster *'Level Control'* (CID: 0x0008) has an attribute called *'current_level'*, which indicates the current output level of the device. A command called ***MoveToLevel*** can set the value of this attribute. Each command can also be called as a **message** and is identified by a command ID (cid). ZCL clusters can be divided into **in-clusters** and **out-clusters**, which define the commands that the endpoint can receive and send respectively.

2.2 Motivation

Table 1 shows the comparison between ZVDetector and existing methods. Protocol state awareness, device state awareness and combined state fuzzing correspond to our three motivations.

• **Motivation 1**: As shown in **Figure 1-CVE1**, a device receiving a specific message or a message sequence may cause an unexpected protocol state transition and trigger a firmware vulnerability. Most studies [35, 36, 40, 41, 48] focus on fuzzing individual protocol messages, ignoring the impact of multiple message combinations on protocol states and the relationship between messages. One type of protocol state fuzzing research [21, 31] tests all possible message combinations. However, due to the large number of Zigbee messages, the combination space explodes. Another type of protocol state fuzzing research [25, 28, 30] learns protocol state models by analyzing RFC documents or running the protocol and then uses these models to guide fuzzing. However, these methods can only generate message sequences to break message dependencies and ignore the effect of correlations between messages. LLMIF [47] applies large language model (LLM) to analyze the message description. However, it can only analyze the impact of the read&write

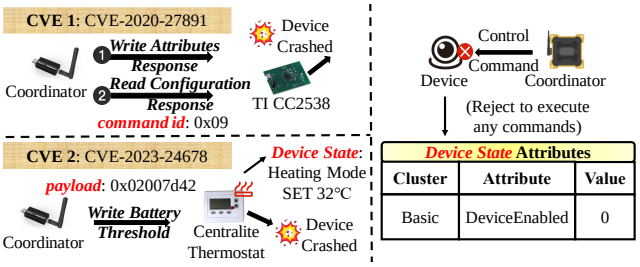


Figure 1: Two CVE vulnerabilities related to protocol state and device state, and an example of internal device state affecting function execution.

correlation on protocol state transition. They fail to completely capture various relationships between messages, especially the correlations that exist in different scenarios. Therefore, these methods either fail to explore the protocol state efficiently or explore only a **limited protocol state space**, restricting the types of message sequences (seen as *actions*) that can be used to trigger vulnerabilities.

• **Motivation 2**: Device state can be seen as the value of all the attributes inside the device at a certain moment. It has a significant impact on the operation of Zigbee stack implementation and affects the correct execution of its functions, as shown in the right part of **Figure 1**. But most studies [21, 25, 28, 36, 40, 41, 47, 48] ignore this point. HubFuzzer only collects the state attributes of in-clusters defined in the Home Automation (HA) scenario by statically analyzing specification and explores only attributes with write permissions. However, it fails to acquire manufacturer-specific attributes and numerous hidden attributes that are not defined in clusters, while also ignores the exploration of many attributes with other permissions. Therefore, these methods overlook the discovery of device state attributes or only explore a **limited device state space**, missing many *preconditions* that could trigger device vulnerabilities.

• **Motivation 3**: A specific message or message sequence under certain device states (*preconditions + actions*) can cause the device to crash, as shown in **Figure 1-CVE2**. So Zigbee protocol states and device states are not independent of each other. Therefore, exploring the **product space** of protocol states and device states is necessary to discover device vulnerabilities, **which has been ignored in most previous studies**. Although HubFuzzer performs state combination black-box fuzzing, it only considers corner case settings of the device state and mutates single protocol messages, thus exploring only the tip of the iceberg of the state combination space. Furthermore, it overlooks the correlation between these two states and thus fails to efficiently explore the combined state space.

2.3 Threat Model and Scope

Threat Model. Our goal is to help administrators and manufacturers identify vulnerabilities in Zigbee devices, thereby enhancing the security of firmware and protocol stack implementations. The discovered vulnerabilities may be exploited by attackers to disrupt the correct execution of functions, even leading to denial of service and bricking the device. Our threat model includes the developer scope and the attack model, with the fuzzer impersonating the coordinator to communicate with the end device.

(1) Developer Scope: The user manually initiates the commissioning (pairing) process according to the device specifications, typically by holding down a specific button on the device for a period of time and establishing a connection with the coordinator. During the communication phase, the coordinator can use the network key obtained during the pairing phase to decrypt messages for collecting device information and sends mutated messages to the end device to perform fuzzing.

(2) Attack Model: The attacker's device must be within the Zigbee network of the target device since Zigbee devices have limited communication ranges, while remote attacks require the device to be connected to a cloud platform [34]. Besides, the end device must be in communication with another coordinator, as attackers generally cannot physically access the end device to initiate the pairing process. The attacker can sniff the original communication traffic to obtain the MAC address of the original coordinator, the MAC address of the target device and the PANID. The EPID is included in the beacon response after sending a beacon request to the original coordinator. Thus, the attackers can disguise themselves as the original coordinator and perform fuzzing attack to the end device. Moreover, the attackers do not need to decrypt Zigbee packets, as they can assume that the device supports all clusters and state attributes, and try to send various messages for the attack.

Scope. Our method is applicable to the vast majority of Zigbee devices, as most manufacturers have joined the Zigbee Alliance and follow the protocol specifications to make their devices. More importantly, our approach **does not require any firmware source code** of the target device for testing or attack, but only a coordinator compatible with the radio type of the end device. Our method requires access to development documentation. Without the open-source documentation, the system can still execute properly by just skipping it. We focus on two vulnerability triggering scenarios in device, including improper sanitization of invalid messages and inadequate validation of legitimate states when receiving messages.

2.4 Challenges and Solutions

(CH-1) Sufficient protocol state space exploration. To discover more vulnerabilities, it is necessary to explore a broader range of message sequences that could trigger unknown protocol state transitions. Existing black-box fuzzing methods [21, 31] attempt to address this by testing all possible message permutations. However, the sheer number of Zigbee messages (assume n) leads to the message arrangements explosion ($\sum_{k=1}^{n} \frac{n!}{(n-k)!} \approx O(n!)$).

Message relationship-based exploration can significantly reduce time overhead. However, Zigbee messages exhibit complex relationship types and interaction scenarios, and incomplete relationship discovery will reduce state space coverage. Existing protocol state learning methods [23, 25, 28] identify message dependencies by analyzing RFC documents or actively running the protocol. However, two messages may affect the common device properties to form a correlation relationship. These approaches cannot find this hidden relationship, as they are neither defined in documentation nor directly observable in runtime traffic. Furthermore, static analysis often fail to capture many such correlations, since many common-effect attributes are hard to determine. Finally, these relationships also manifest in multi-device interaction scenarios, where static analysis struggles to verify their validity.

✓ **Solution 1**: We find that LLM can more comprehensively find the device attributes affected by each message, while active probing can collect message dependencies and verify the message correlation. Therefore, we propose a technique that combines static and dynamic analysis to more thoroughly identify message relationships **(§4.2)**. Moreover, we design a heuristic relationship-based state graph exploration method that can efficiently explore unknown protocol state transitions **(§4.3)**.

(CH-2) High device state attributes coverage. First, it is essential to fully understand the state attributes contained in the device. The device internally includes a large number of basic state attributes defined in clusters. However, some of these attributes are set with query isolation, and many others are customized attributes defined by manufacturers when adding unique functions to the device. These two types of attributes cannot be discovered through active cluster queries. Additionally, the device contains numerous hidden state attributes that are not defined in clusters. These attributes cannot be discovered through active cluster queries or by sending Read/WriteAttributes messages.

Secondly, it is necessary to thoroughly explore the device state space. However, many attributes have complex data types, such as variable-length array, making it difficult to modify and read attribute values. Even worse, most manufacturers set many attributes to **ReadOnly** access for security considerations, disabling ***WriteAttributes*** commands, which significantly increases the difficulty of changing attribute values. Existing specification-based static analysis methods [35] can only partially obtain basic state attributes and parse common attribute data types, and it fails to understand how the ReadOnly attributes can be modified.

✓ **Solution 2**: Due to the complexity of the state attributes, we find it difficult to analyze them completely by just one method. Therefore, we propose a three-stage multi-source data mining approach that combines active querying, specification and manufacturer development document analysis to completely discover basic state attributes and parse their data types, and a LLM-based message format analysis technique to discover hidden state attributes **(§5.1)**. We find that many ReadOnly attributes can be modified by certain messages when analyzing the impact caused by messages. Based on this insight, we use LLM to infer the message-writable attributes**(§5.2)**.

(CH-3) Efficient combined state space fuzzing. While exploring the combined state space using the idea of black-box fuzzing [35] can achieve complete coverage, the time overhead for traversal is huge, and many invalid cases exist. Therefore, it is imperative to design a fuzzing algorithm to efficiently explore the combination space and generate test cases that can quickly reach firmware vulnerabilities.

✓ **Solution 3**: To reduce the invalid test cases, we design two message semantic-aware mutation strategies: format-aware mutation and type-aware mutation. To efficiently explore the state combination space, we develop two optimization schemes in the fuzzing algorithm. The first is to analyze the correlation between the two states, focusing on the device state attributes that are closely related

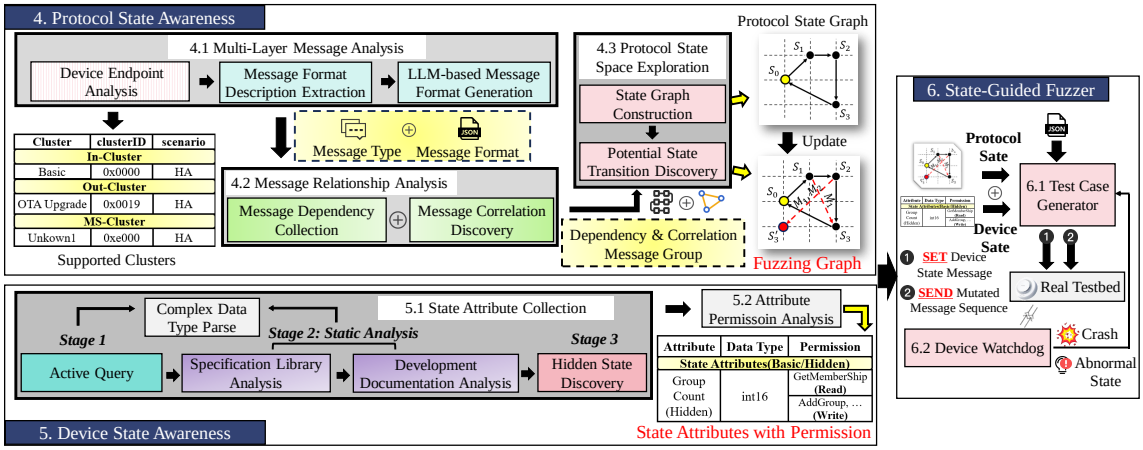


Figure 2: The system architecture of ZVDetector.

to the message sequence under test. The second is a crash feedback-based adjustment mechanism. The algorithm focuses on other test cases near the combined state that caused the crash (**§6.1**).

3 System Architecture

Figure 2 shows the three key modules of ZVDetector.

- **Protocol state awareness module (§4).** First, we analyze the clusters supported by the device endpoints using an active probing method. Then, we develop an LLM-based message description analysis method to acquire multi-layer message formats and filter out unsupported messages **(§4.1)**. To address **CH-1**, we design a method that combines static and dynamic analysis to discover various message dependencies and correlations **(§4.2)**. Moreover, we propose a heuristic relationship-based graph exploration method that efficiently identifies more message sequences leading to unknown protocol state transitions **(§4.3)**.
- **Device state awareness module (§5).** We design a three-stage attribute collection method that combines dynamic active querying, static document analysis, and an LLM-based message analysis approach to collect both basic and hidden state attributes **(§5.1)**, addressing the issue of incomplete state attribute collection in **CH-2**. Besides, we develop a LLM-based attribute permission analysis technique **(§5.2)**. With the complex data types parsing during attribute collection, we achieve finer-grained understanding of state attributes. This enables us to determine the read and write methods for each state attribute, tackling the challenge of insufficient device state space exploration in **CH-2**.
- **State-guided fuzzer (§6).** Since the combined state space is large, an efficient fuzzing approach is needed to address the challenges in **CH-3**. We design a state-guided fuzzing algorithm to generate test cases **(§6.1)**. The state-guided part of the test case generator uses the fuzzing graph that contains the protocol states to generate the message sequence for testing and analyzes the device state associated with each message sequence. The mutation and testing part uses the format-aware mutation strategies to send correlated device state setup messages and mutated protocol messages to the device and adjusts the test cases based on the device crash or anomaly state fed back by the device watchdog **(§6.2)**.

4 Protocol State Awareness

In this section, we first obtain message descriptions for different protocol layers. Then, we utilize active probing and LLM-assisted static message analysis to discover message relationships. Finally, we explore unknown protocol state transitions within the constructed state graph based on these message relationships.

4.1 Multi-Layer Message Analysis

With the pretrained LLM, we generate the complete format for each message based on their descriptions extracted from documentation. These formats not only help in analyzing message relationships **(§4.2)** and device state attributes writability **(§5.2)**, but also guide the fuzzer in performing precise message mutations **(§6.1)**.

4.1.1 Device Endpoint Analysis. ZCL messages and attributes are defined under the clusters of each endpoint. Therefore, it is necessary to first determine which endpoints the device supports and which clusters they contain. We send an ***ActiveEndpointRequest*** message to the device to collect all supported endpoint IDs and then send a ***SimpleDescriptorRequest*** message for each endpoint to gather the supported clusters. The ***SimpleDescriptorResponse*** message contains two types of clusters supported by the device: input clusters (in-clusters) and output clusters (out-clusters). Some clusters are presented as 'Unknown' since they are manufacturer-specific rather than common-used defined in the specification.

The following is a description of a **Zigbee {layer}** **{frame_type}** frame **{header/payload}** format. **{format_description}**

Please extract the format of this frame.

The answer should be organized in **JSON** format. Following is an example of ZDP Command **SimpleDescriptorRequest** payload format.
{"SimpleDescriptorRequest":{
"Nwk Addr Of Interest": "uint16",
"Endpoint": "uint8"}}

Message Type	Message Format
MAC Command/ACK	$F_{C_i}^{MAC}/F_{Ack}^{MAC}$
NWK Command	$F_{Data}^{MAC}+F_{C_i}^{NWK}$
APS Command/ACK	$F_{Data}^{MAC}+F_{Data}^{NWK}+F_{C_i}^{APS}/F_{ACK}^{APS}$
ZDP Command	$F_{Data}^{MAC}+F_{Data}^{NWK}+F_{Data}^{APS}+F_{C_i}^{ZDP}$
ZCL Command	$F_{Data}^{MAC}+F_{Data}^{NWK}+F_{Data}^{APS}+F_{C_i}^{ZCL}$

Annotations

$F_{\{Frame\}}^{\{Layer\}}$: Header format of {Layer} {Frame (Data/ ACK)}

$F_{C_i}^{\{Layer\}}$: Payload format of {Layer} command C_i

Figure 3: The LLM prompt for multi-layer message generation and the methods for concatenating message formats.

4.1.2 Message Format Description Extraction. We first analyze the ZCL-layer message format, as it can be obtained from the Zigpy

[17] library directly. Zigpy strictly implements the Zigbee Alliance specification. Most devices follow this specification when implementing the protocol stack in their firmware. The in-clusters and out-clusters messages are defined in two separate inner classes *ServerCommandDef* and *ClientCommandDef*, containing the message name, message ID, and a schema that specifies the payload format. Similarly, the ZCL general command can be acquired from *GeneralCommand* class and the header format can be retrieved from the *ZCLHeader* class. The extracted part of the Zigpy library code is described in **Appendix A**. We denote the format of ZCL command C_i as $F^{ZCL}_{C_i}$.

The message format descriptions for other layers are all defined in the protocol specifications: the MAC layer is specified in the IEEE 802.15.4 standard [8], while the APS, NWK layer and ZDP are defined in the Zigbee Alliance specification [15]. The specifications include various message frame types, and we extract the structural descriptions of each frame type from the chapter *"Format of Individual Frame Types"*. Specifically, *ACK* frames do not contain a payload. The payload of lower-layer *Data* frames contains higher-layer messages, so we only extract its header part. We separately extract the payload descriptions of *Command* frames, including command/cluster ID, name and illustrations of all fields, as they are defined in other sub-chapters, such as *"Command Frames"*. We also extract each ZCL message description from the ZCL Guide [13] based on cid and CID for the attribute permission analysis (**§5.2**).

4.1.3 LLM-based Message Format Generation. Based on the message descriptions, we design a LLM prompt to generate the format for each part of a message, as shown in **Figure 3**. For *Data* and *ACK* message frames, the prompt directly generates their header formats. For *Command* frames, the prompt generates the payload format for each command *Ci*. Finally, we combine all parts to construct the complete message format and filter out unsupported messages by actively sending them and checking the response.

4.2 Message Relationship Analysis

Apart from effecting device attributes during execution, one message may also be related with another. Specifically, we categorize the message relationship into two types: **dependency** and **correlation**. Dependencies exist between multiple messages in a strict order, while correlations exist between multiple messages that have a common effect on a device. Next, we describe our combined static and dynamic analysis approach to address the insufficient message relationship discovery mentioned in **CH-1**.

4.2.1 Message Dependency Collection. We dynamically send each message at fixed intervals and analyze the traffic. One type of dependency forms a **causal relationship**, where triggering one message leads to the occurrence of another. If the traffic within the interval contains a response/APS ACK message, we consider them to be causally related. Another type of dependency involves **ordering constraints**, where messages must be executed in a strict sequence (handshake phase). We set the interval at 1 second since most request-response interactions complete within 0.1 seconds and there are hardly more than 10 interactions included in a sequence.

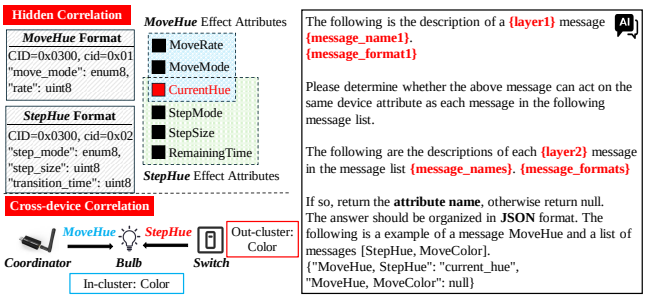


Figure 4: An example of hidden correlation and cross-device correlation. Hidden correlations are acquired using LLM prompts described on the right side.

Moreover, with the help of **§5.2**, we distinguish finer-grained protocol states by identifying the attributes modified by messages and reading the values of these attributes through certain messages.

4.2.2 Message Correlation Discovery. Correlation exists between two messages that act on the same attribute. We find such correlations through three strategies. **(1)** Common fields in their payloads. **(2)** Field containment relationships, which can be determined by their data types. For example, the message ***GetMembership*** has field *'groups'* with the data type *'List[Group]'*. Another command ***AddGroup*** has field *'group_id'* with data type *'Group'*. We can ensure that *'groups'* contains *'group_id'*. These two types form the basic correlations. **(3) Hidden Correlations:** Two messages may act on the same attributes, but not be reflected in their format. **Figure 4** shows two messages that belong to this correlation type. We construct LLM prompts to determine whether a message will form a correlation with other messages by analyzing message formats. If the correlated attributes are not in the list of attributes collected in **§5.1**, we will add them as hidden attributes.

Besides, correlations may exist in multi-device interaction scenarios. An end device may receive messages from another device while simultaneously communicating with the coordinator. The following conditions must be met: one device D_A has an out-cluster, and the same cluster type exists in the in-clusters of another device D_B. However, as discussed in **CH-1**, static analysis is hard to verify their authenticity, since many low-power and vendor-specific devices may disable control from third-party devices or only support reporting their own status. So we design a dynamic verification method. By sending two ZDP messages ***BindRequest*** and ***DisplayBindTable***, we determine whether the in-cluster of D_B can be bound by the D_A. Then, we send out-cluster requests from D_A to D_B and check the response is generated, such as APS ACK.

4.3 Protocol State Space Exploration

Unknown protocol states transition may be triggered by certain messages. However, blindly searching the message combination is unacceptable, as discussed in **CH-1**. Therefore, we next describe how to efficiently explore unknown protocol states by using message relationships, which have been sufficiently mined before.

4.3.1 State Graph Construction. Similar to existing protocol state fuzzing methods, we construct the basic state transitions based on message dependencies. However, we differentiate between the pairing phase and the communication phase by setting distinct initial state nodes. Once the pairing phase is complete, the initial node

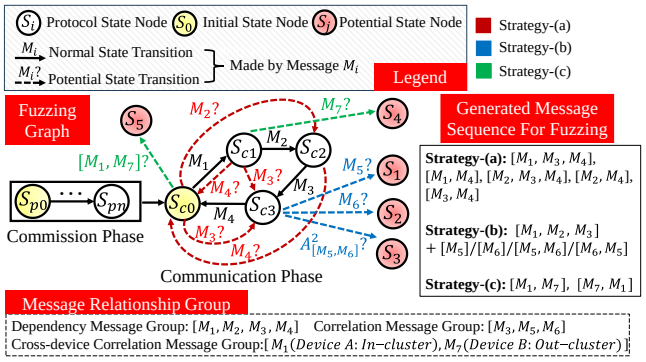


Figure 5: Three strategies are used to discover potential protocol state transitions on the basic protocol state graph.

of the communication phase will be reached, where we consider the message ***DeviceAnnouncement*** as the result of pairing. Furthermore, after a message sequence with dependencies has been executed, the protocol state returns to the initial communication state node, since its execution result does not affect other messages.

4.3.2 Potential State Transition Discovery. First, we design the following three exploration strategies for different relationships, which can fully explore the unknown state transitions guided by message sequences that form each relationship type.

(a) Break Dependency Skipping: We test whether it is possible to skip certain messages within a dependent message group, thereby disrupting the original strict order. As shown in **Figure 5**, we attempt to skip message M_1 by directly sending message M_2 (red edge) to see if we can reach state S_{c2} directly. This prevents the device from receiving critical information on certain messages exchanged in the normal process and leads to a unexpected state.

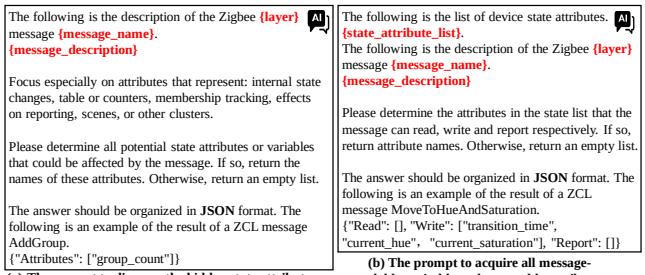
(b) Correlation Effect Insertion: We attempt to insert other messages correlated with one of the messages in a dependent sequence, such as inserting messages M_5 and M_6 (blue edge) after the execution of message M_3, as shown in **Figure 5**. Our insight is that correlated messages may alter the execution result of the original sequence and influence the device to process subsequent messages.

(c) Out-cluster Aware Insertion: Control messages from other devices may also alter the execution result of a device's message sequence. Based on the cross-device correlation found previously, we insert out-cluster messages from other devices before and after the correlated in-cluster messages, such as inserting out-cluster message M_7 (green edge) from device B before and after in-cluster message M_1 from device A in **Figure 5**.

Secondly, a **basic strategy** is designed to explore all single edges of the state graph, as they may trigger potential state transitions when combined with mutation strategies. We refer to the state graph updated by these strategies as the fuzzing graph G_{fuzz}.

5 Device State Awareness

Device state can be seen as the combination of all state attribute values. In this section, we propose a multi-source data mining approach to collect various device state attributes and combine it with LLM to analyze the writability of these attributes. This enables a fine-grained perception of device states and addresses **CH-2**.



(a) The prompt to discover the hidden state attributes. **(b) The prompt to acquire all message-readable, writable and reportable attributes.**

Figure 6: Two LLM prompts used to discover hidden state attributes and analyze the attribute permissions.

5.1 State Attribute Collection

Specifically, the state attributes supported by the device can be classified into three categories: (1) Explicitly defined in the cluster and can be directly queried. (2) Explicitly defined in the cluster but set with query isolation. (3) Not explicitly defined in the cluster. The first two kinds form the basic device state space D^{state}_{basic}, while the third type constitutes the hidden state space D^{state}_{hidden}. To fully cover each attribute type, we design the following three-stage approach.

5.1.1 Stage1: Active Query. We query the attributes supported by each cluster, along with their data types and access permissions, by sending ***DiscoverAttributes*** and ***DiscoverAttributesExtended*** messages. However, for security reasons, many devices refuse to respond to these two messages or only return a subset of attributes.

5.1.2 Stage2: Static Analysis. The second type of state attributes can be further divided into common-used attributes and manufacturer-specific (MS) attributes. The former can be analyzed through the specification library, while the latter can be acquired from the manufacturer's publicly released development documentation.

Specification Library Analysis. Similar to **§4.1.2**, we extract attribute IDs, data types, and access permissions from the inner class *AttributeDefs* under each cluster class defined in the Zigpy. The permissions include Readable (R), Writable (W), and Reportable (P). Besides, we recursively parse the datatype class of each attribute to obtain the structure and fields of complex data types. Finally, we send ***Read/WriteAttributes*** or ***ConfigureReporting*** messages to filter the unsupported attributes according to their permissions.

Development Documentation Analysis. Many MS attributes are hard to discover, read, and modify since manufacturers not only set query restrictions but also introduce unique semantics, making them have complex data structures. For example, the attribute *'Biorhythm'* defined in the Tuya Bulb has a variable-length structure *'Array'*, where one byte's low-high bits represent the day of the week. We find that MS commands can modify MS attributes. Therefore, we first ensure the mapping relationships via the link symbols and bind each attribute to its corresponding command. Then we extract each field structure in the command format tables, which contains different meanings, and combine them as the attribute data type. The extraction details are described in **Appendix B**.

5.1.3 Stage3: Hidden State Discovery. For the third attribute type, we only consider hidden state attributes that can be accessed through commands, as others cannot be modified or read. For example, a hidden attribute is *'group list'*, which can be known by the message

Algorithm 1: Fuzzing Algorithm(State-Guided Part).

procedure: $M_s, D^{state}_{M_s} \leftarrow \text{SG}(D^{state}_{all}, G_{fuzz})$
1 $M_s \leftarrow [], D^{state}_{M_s} \leftarrow \phi$
2 $M_s \leftarrow Select\_Path(G_{fuzz}, S_{basic})$
3 **if** *not* M_s **then**
4 $\quad M_s \leftarrow Select\_Path(G_{fuzz}, S_a)$
5 **if** *not* M_s **then**
6 $\quad M_s \leftarrow Select\_Path(G_{fuzz}, S^{comm}_{init}, S_?)$
7 **for** M_i *in* M_s **do**
8 $\quad D^{state}_{M_i} \leftarrow Corr\_State(M_i, D^{state}_{all})$
9 $\quad D^{state}_{M_s} \leftarrow D^{state}_{M_s} \bigcup D^{state}_{M_i}$
10 **return** $M_s, D^{state}_{M_s}$ ▷ $len(M_s) \geq 1$

GetMembership and can be modified by the message ***AddGroup*** etc,. Typically, these attributes are reflected by the functionality of messages. As shown in **Figure 6(a)**, we design this LLM prompt to analyze the message descriptions, as LLM has a better understanding of text semantics. We get a list of possible hidden attributes and remove those that are duplicated with the basic attributes D^{state}_{basic}.

5.2 Attribute Permission Analysis

The device state attributes D^{state}_{all} can be represented as the union of D^{state}_{basic} and D^{state}_{hidden}. However, many state attributes can only be queried but cannot be modified, mutating these attributes only results in unnecessary time overhead. Besides, many attributes are set with **ReadOnly** permission. However, We find these attributes simply disable the message ***WriteAttributes***, but can still be modified by other messages. Therefore, we analyze the attributes that each message can modify using the LLM prompt shown in **Figure 6(b)** and filter out all non-writable attributes from D^{state}_{all}. Additionally, we analyze the attributes that each message can read and report since the device state needs to be acquired during fuzzing, while a few attributes cannot be read via ***ReadAttributes*** directly. Moreover, subsequent fuzzing needs to determine the attributes correlated with each message, and this correlation can be seen as all attributes that a message can either query or modify.

6 State-Guided Fuzzer

In this section, we describe how to generate test cases of combined states to explore vulnerabilities in device firmware. To discover that vulnerabilities are triggered, we design device watchdogs to monitor the occurrence of crashes and abnormal state events.

6.1 Test Case Generator

To efficiently explore the product space of device states and protocol states, thereby solving the **CH-3**, we design a grey-box fuzzing algorithm to generate test cases. Specifically, the algorithm can be divided into the state-guided part and the mutation part.

6.1.1 State-guided part. In this part, the perceived state information will be used to generate seeds that guide subsequent mutations. The detailed pseudo-code is shown in **Algorithm 1**. First, a message sequence M_s will be determined to be fuzzed based on the fuzzing graph G_{fuzz} obtained from the protocol awareness module. The algorithm prioritizes the basic strategy to explore all single message edges in the protocol graph **(line 2)**. Then, it firstly tracks skipping dependency paths updated by strategy **(a) (line 4)**. The messages contained in each edge of the selected path are combined to form the message sequence M_s. After that, the algorithm explores paths from the initial communication state node S^{comm}_{init} to the potential state nodes $S_?$ **(line 6)**, which include protocol state transition paths updated by strategies **(b)** and **(c)**.

Correlated Device State Analysis. While the combination of traversing state attributes provides a complete exploration of the device state space, most device states are not related to specific messages and only increase the exploration cost during fuzzing and the time cost to set these unrelated states. To address this, we use the results of LLM prompt analysis in **§5.2** to obtain the device state attributes $D^{state}_{M_i}$ correlated with each message M_i in the message sequence M_s **(line 8)**. Finally, they are combined into a complete device state $D^{state}_{M_s}$ associated with the message sequence M_s **(line 9)**, which will be mutated and set to the device later.

Algorithm 2: Fuzzing Algorithm(Mutation & Testing Part).

procedure: $L_{interest} \leftarrow \text{Fuzz}(M_s, D^{state}_{M_s}, G_{fuzz})$
1 $M^{fuzz}_s \leftarrow Mutate(M_s), D^{fuzz}_{M_s} \leftarrow Mutate(D^{state}_{M_s})$
2 **for** $D^{fuzz}_{M_i}$ *in* $D^{fuzz}_{M_s}$ **do**
3 $\quad Set\_Device\_State(D^{fuzz}_{M_i})$
4 $\quad$ **for** M^{fuzz}_i *in* M^{fuzz}_s **do**
5 $\quad\quad F_{crash}, F^{state}_{abnormal} \leftarrow Send(M^{fuzz}_i)$
6 $\quad\quad M^{fuzz}_s.remove(M^{fuzz}_i)$
7 $\quad\quad$ **if** F_{crash} *or* $F^{state}_{abnormal}$ **then**
8 $\quad\quad\quad L_{interest}.append(D^{fuzz}_{M_i}, M^{fuzz}_i)$
9 $\quad\quad\quad M^{fuzz}_{new} \leftarrow Similar(G_{fuzz}, M^{fuzz}_i)$
10 $\quad\quad\quad M^{fuzz}_s = [M^{fuzz}_{new}, M^{fuzz}_s]$, *GoTo line* 3
11 **return** $L_{interest}$

6.1.2 Mutation & Testing part. In this part, we mutate the seeds obtained from the previous part and conduct fuzz testing on the device. Additionally, we adjust the test cases based on feedback results, allowing the algorithm to focus on areas near device vulnerabilities. The pseudo-code is shown in **Algorithm 2**. Specifically, we first design two semantic-aware mutation strategies:

(1) Type-aware mutation. (a) Border values and extreme values for integers or floating-point numbers: We generate hexadecimal values equal to or exceeding the maximum and minimum values based on the number of bytes occupied by their data types, as well as perform sign flipping. (b) Empty or excessively long strings: Empty strings can trigger null pointer references, while overly long strings can cause buffer overflows. (c) Empty arrays: Some manufacturer-specific attributes with an *'Array'* structure may trigger null pointer references when their values are empty. (d) Type changes: We change array and integer types to strings, unsigned integer to signed integer, and convert strings into character arrays.

(2) Format-aware mutation: (a) Truncated messages: Truncate all bytes in the APS layer or ZCL layer of the message. A complete MAC and NWK layer may cause the coordinator or device to continuously process meaningless messages. (b) Add/ remove fields or increase the number of arguments in the ZCL layer. (c) Provide unsupported arguments: these unsupported arguments include cluster IDs, attribute IDs, and command IDs in the APS and ZCL layer.

Before the fuzzing starts, a snapshot of the device state is first obtained. For readable and reportable device state attributes, the values are read directly. For other writable attributes, they are set to their initial values. We keep a snapshot of the initial device state D_0, and a snapshot of the current device state D_{now}.

Combined State Fuzzing. The fuzzer first generates a list of mutated message sequence M_s^{fuzz} and a list of mutated state dictionaries $D_{M_s}^{fuzz}$ based on these strategies **(line 1)**. Then it combines a mutated message sequence M_i^{fuzz} and a mutated device state dict $D_{M_i}^{fuzz}$ from the two lists to generate a test case. **Device State Setup**: The fuzzer sets the mutated state in the device **(line 3)** by sending the message ***WriteAttributes*** or other writable commands(analyzed in **§5.2**) to modify the attributes. Instead of resetting all state attributes, which will waste a lot of time since many state attributes keep the same, we compare the current state snapshot with the new device state(to be set) and only modify those state attributes with different values. After the device state settings are complete, the fuzzer sends the mutated message sequence to the device **(line 5)**.

Crash-based Feedback Adjustment. When a device crashes, the test case that triggered the crash will first be stored, including the message sequence M_i^{fuzz} and a snapshot of the current device state $D_{M_i}^{fuzz}$. Then, the algorithm explores the combined state space near the test case, expanding the test case by searching for similar message sequences M_{new}^{fuzz} **(line 9)** and add to the front of all test cases **(line 10)**. The criteria for determining similarity include containment relationships or the presence of two or more identical message types. After that, we first manually reboot and repair the device. Then we reset the device by continuously pressing the reset button, thus the device can be restored to its initial state ($D_{now} = D_0$). If neither rebooting nor resetting restores the device to normal function, we consider it to be bricked. Then the algorithm fixes the device state that triggered the crash and starts a new round of fuzzing **(line 10 → line 3)**. Finally, we obtain all test cases $L_{interest}$ that cause crashes or trigger abnormal states.

6.2 Device Watchdog

During normal communication, the device D has the dependency on its interaction with the coordinator C, namely, Request ($C \rightarrow D$) - Ack ($D \rightarrow C$) - Response ($D \rightarrow C$) - Ack ($C \rightarrow D$). When a device **crash** occurs, all acknowledgments from the device to the coordinator will not be generated. Therefore, the watchdog only needs to check whether the device sends an Ack after a request. An **abnormal state** of the device occurs when the device is still alive (reply Ack for request) but unable to generate response messages to normal control commands. The watchdog determines this by sending commands ***'On/Off'***.

7 Evaluation

In this section, we present the experimental results to answer the following three questions corresponding to the **CH-1** to **CH-3**:

Q1: Can ZVDetector accurately generate message formats of each layer and discover message relationships more sufficiently?

Q2: Can ZVDetector collect more device state attributes with their permissions and parse complex data types of attributes?

Q3: Can ZVDetector efficiently explore product state space and detect vulnerabilities triggered by combined states?

7.1 Experimental Setup

Table 2: Ten devices deployed in testbed.

ID	Type	Vendor	Device Model	ID	Type	Vendor	Device Model
1	Plug	Aqara	lumi.plug	6	Bulb	Sengled	E11-N1EAW
2	Switch	Tuya	SM0004	7	Strip	Sengled	E1G-G8E
3	Plug	Tuya	TS011F	8	Valve	Tuya OMEG	TS0049
4	Bulb	Tuya	TS0505B	9	Bulb	Third Reality	3RCB01057Z
5	Switch	Sengled	E1E-G7F	10	Plug	Third Reality	3RSP02028BZ

Testbed. We select ten Zigbee devices for testing, including five different types: switch, plug, bulb, strip and valve, covering five well-known manufacturers: Lumi Aqara, Tuya, Sengled, Tuya OMEG and Third Reality, as shown in **Table 2**. To connect these Zigbee devices, we use two Zigbee gateways as the coordinators: the SONOFF Zigbee 3.0 USB Dongle Plus (DongleP) and the Nortek GoControl QuickStick HUSBZB-1 (HUBZ), which connect to the PC via USB ports. A snapshot of testbed is shown in **Appendix D**.

Implementation Configuration. ZVDetector runs on Ubuntu 20.04 virtual machine (VM) with a 2.6 GHz six-core Intel Core i7 CPU and 16GB RAM, where the gateway is DongleP. Additionally, we build Home Assistant 2024.4.2 on another Ubuntu 20.04 VM with the same configuration and install the Zigbee Home Automation (ZHA) integration to build another Zigbee network, where the gateway is HUBZ. Additionally, a CC2531 USB Dongle is connected to a Windows 11 machine with an i5 CPU and 16GB RAM, running the TiWsPc [16] tool to capture Zigbee traffic. We use GPT-4o API for format generation and DeepSeek-V3 API for messages and attributes analysis, since they are two SOTA LLMs in the market.

Baseline Model. B1: LLMIF [47] has been proven effective in message format generation and relationship discovery **(Q1)**. We construct the same LLM prompts used in LLMIF. We also integrate LLMIF with our message mutation strategy to perform protocol state fuzzing **(Q3)**. **B2:** Another protocol state fuzzing method proposed by Fiterau et al. [28, 30], which follows the DTLS-Fuzzer [29] framework. We implement their model learning using LearnLib [9] to collect message dependencies **(Q1)**, and change the fuzzing method of TLS-Attacker [12] to adapt to Zigbee **(Q3)**, as its mutation strategy is tailored to the TLS characteristics. **B3:** Beehive [48] is effective in vulnerability discovery using single protocol message fuzzing **(Q3)**. We implement its mutation method and perform semantic-aware fuzzing based on our extracted message formats. **B4:** HubFuzzer [35] is proved to be effective in attribute collection **(Q2)** and combined state fuzzing **(Q3)**. We implement a prototype of HubFuzzer since the code is not available in its open-source project.

Table 3: The comparison of the correctly generated message formats and discovered relationships. [C] represents the non-MS ZCL cluster messages, [M] represents MS ZCL messages and [G] represents the ZCL general messages. [B], [H], [O] represents basic, hidden and cross-device correlations.

Number / Class	Method	B2	B1	ZVDetector
Format	ZCL Command	N/A	220	382 (279[C] 83[M] 20[G])
	ZDP Command	N/A	N/A	85
	APS Command/ ACK	N/A	N/A	10/1
	NWK Command	N/A	N/A	13
	MAC Command/ ACK	N/A	N/A	9/1
	Total	N/A	220	501
Relationship	Dependency	325	N/A	366
	Correlation	N/A	417	904[B] 2246[H] 612[O]

7.2 Protocol State Awareness Effectiveness (Q1)

Message Format Generation Performance. We actively send the supported messages to device and capture the traffic to extract each message format as the ground truth. As shown in **Table 3**, B1 achieves only 78.85% accuracy in format generation for 279 ZCL Non-MS cluster-specific messages of 27 clusters, since some message descriptions are coarse-grained and lack enhanced field descriptions, whereas ZVDetector achieves accurate format extraction for all these messages. Besides, we successfully extract the formats of 83 ZCL MS messages and 20 ZCL general messages, and 119 formats from other six message types. B1 fails to extract the formats from layers other than ZCL due to the heterogeneous description structures. Other details are listed in **Appendix C**.

Discovered Message Relationships. Compared to B2, we discover 41 more dependencies. Although LearnLib defines an input symbol set that includes all messages, it fails to distinguish some fine-grained states. For example, the message ***toggle*** switches the device between on and off states, but the default responses are the same with 'status': 'success'. Our approach reads the state attributes correlated with messages, and thus successfully constructs two separate state transition branches instead of one.

Besides, we identify and validate a total of 3,150 correlations, including 904 basic correlations and 2,246 hidden correlations with no ACK& Data frame messages, among which 612 also fit the cross-device correlation scenario. We first construct a list of non-correlated format fields, including various identifiers and generic addresses, to filter out 23,906 invalid basic correlations and 2,719 invalid hidden correlations. Then we check all hidden correlations that are duplicated with the basic correlations. We find 25 duplicate hidden correlations, but 21 of those find new correlated attributes. Finally, we validate all correlated attributes by determining whether the correlated messages can read, write, or report these attributes and find the remaining 8.1% of LLM results to be invalid. **Read:** The message is executed and we check whether the response traffic includes the attribute and its value. **Write:** The message is executed, and another message capable of reading the attribute (acquired in **§5.2**) is used to verify whether the value has changed. Few attributes are non-readable by messages. We directly assume that the message can modify the attributes. **Report:** We check whether the message directly contains the attribute value while the coordinator can receive it. The validation method for the results of **§5.2** is described in **§7.3**. Compared to B1, we are able to detect a wider range of correlations beyond just read&write operations, including write&report operations and read&report operations.

Table 4: Device endpoint information coverage results.

ID	ZVDetector			HubFuzzer		
	Cluster	Attribute	Command	Cluster	Attribute	Command
1	16(100%)	237(100%)	60(100%)	11(68.75%)	166(70.04%)	40(66.67%)
2	8(100%)	73(100%)	64(100%)	5(62.50%)	44(60.27%)	40(62.50%)
3	13(100%)	357(100%)	102(100%)	9(69.23%)	320(89.64%)	74(72.55%)
4	10(100%)	166(100%)	129(100%)	8(80.00%)	114(68.67%)	86(66.67%)
5	10(100%)	72(100%)	57(100%)	5(50.00%)	42(58.33%)	26(45.61%)
6	9(100%)	117(100%)	58(100%)	8(88.89%)	99(84.62%)	48(82.76%)
7	9(100%)	117(100%)	58(100%)	8(88.89%)	99(84.62%)	48(82.76%)
8	10(100%)	79(100%)	58(100%)	5(50.00%)	44(55.70%)	40(68.97%)
9	8(100%)	137(100%)	79(100%)	7(87.50%)	114(83.21%)	68(86.08%)
10	10(100%)	339(100%)	85(100%)	8(80.00%)	308(90.86%)	74(87.06%)

LLM Time Cost. GPT-4o completes the format generation for 119 non-ZCL messages within 22.81 minutes. For correlation analysis, overly long message lists can lead to attention dilution, while short message list will increases API calls with high time cost. Therefore, we divide the messages at each layer into groups of 20 (batch size=20). DeepSeek-V3 completes 6976 batch rounds in 17.2 hours and analyze 136,082 message pairs. Despite the high time costs, these are one-time efforts and the results can be directly reused.

7.3 Device State Attribute Coverage (Q2)

Endpoint Information Coverage. We compare the discovered clusters, attributes, and commands with B4, as shown in **Table 4**, where we achieve better coverage across all 10 devices. We successfully collect 11 MS clusters and 18 out clusters, while B4 only identifies Non-MS clusters in setting-up messages and ignores all out-clusters within endpoints. Besides, we discover 44 MS attributes and 16 MS attributes are defined under non-MS clusters. We discover and verify 116 hidden attributes by applying the same method as for validating hidden correlations, and 21.05% of results generated by LLM are invalid. Finally, we evaluate the attribute permission analysis performed by the LLM in **§5.2**. We first adopt the above **Read** and **Report** verification methods to determine the correctness of message-readable and message-reportable permissions. Based on the valid readable attributes, we use the above **Write** method to verify all message-writable permissions. The results include 64 message-readable, 67 message-reportable and 295 message-writable attributes, and 15.98% of results generated by LLM are invalid.

To support more devices for future research, we also analyze all 279 messages across 27 commonly used ZCL clusters with a larger D^{state}_{basic} containing 590 attributes and other non-MS and non-ACK 137 messages. We discover 375 hidden state attributes D^{state}_{hidden} and 14,878 attribute permissions. These attributes significantly expand the device state space. Additional results need to be validated on devices that support related messages and attributes.

LLM Time Cost. DeepSeek-V3 analyzes one message per round, taking 9.4 minutes and 35.1 minutes for hidden state attribute analysis on two scales. For attribute permission analysis, we similarly split the state attribute set into groups of 20 to make the LLM more focused. It takes 6.19 hours on the endpoint support scale and 27.77 hours on the larger scale.

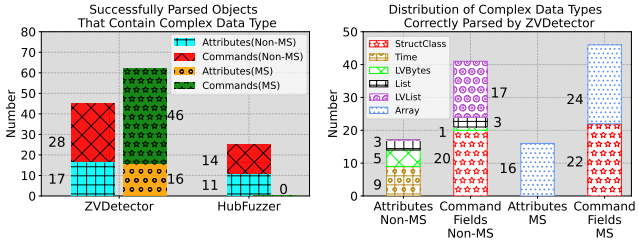


Figure 7: Comparison of successfully parsed attributes and commands containing complex data types with B4.

Complex Data Type Parsed Results. Many attributes and message fields have complex data types, which makes it difficult for the fuzzer to send commands, and for the watchdog to parse them. **Result verification method:** We construct messages containing the attributes based on the collected data types. By sending these messages, we check whether a normal response is received and analyze the structure of the response to determine if each field matches the expected format. As shown in **Figure 7**, B4 successfully parses 25 attributes and commands with complex data types, while ZVDetector successfully parses 107 objects, which contains 120 command fields and attributes with 6 complex data types. B4 fails to parse all complex data types on MS objects and some complex data types on Non-MS objects including 'StructClass' and 'LVList'.

Table 5: Information about the vulnerabilities discovered on the testbed. Ⓓ and Ⓒ represents dependency and correlation group respectively. [F] and [T] represents the format-aware and the type-aware mutation strategy respectively.

Device ID	Exploit Messages	Mutation Method	Observations	CVE	Count
1	**[Out-cluster]**: *BindRequest BindRegisterResponse* Ⓒ	[F] BindRequest: No ZDP frame	Device Crashed	Confirmed	1
2	*DiscoverCommandGeneratedResponse*	[F] No *'discovery_complete'* field	Device Crashed	Under Review	1
3	*OnWithTimedOff*	[T] *'on_off_control'*: Invalid Value *'on_time'*, *'off_time'*: Negative integer overflow	Device Crashed	**New CVE**	2
	[MS-cluster]: *InchingSwitch*	[T] Array field(*'inching_duration'*): Invalid & Negative Value	Abnormal State	Under Review	3
4	*WriteAttributes, WriteAttributesResponse* Ⓓ	[T] *'device_enabled'*: Negative integer overflow	Abnormal State	**New CVE**	4
	AddIfIdentifying, AddGroup, EnhancedAddScene, GetSceneMembership Ⓒ	[T] **Device State**: [group_count: 0] **(Hidden State)** AddGroup: *'group_name'*: String overflow	Device Crashed	**New CVE**	2
	AddGroup, GetGroupMembership Ⓒ	[T] *'group_name'*: String overflow	Device Crashed	Under Review	2
5	*SetShortPollInterval*	[T] *'New_short_poll_interval'*: 0x0000/Negative integer	Device Crashed	Old CVE	2
6	*StepLevel, MoveLevel* Ⓒ	**Device State**: [Current Level: 0xfe] StepLevel: *'step_mode'*: 0x01 *'step_size'*: 0xff MoveLevel: *'move_mode'*:0x00 *'move_rate'*: 0x00	Device Crashed	**New CVE**	1
	StepLevel, MoveLevel Ⓒ	[T] **Device State**: [Current Level: 0x00] StepLevel: *'step_mode'*: 0x00 *'step_size'*: 0xff MoveLevel: *'move_mode'*: 0x01 *'move_rate'*: 0x00	Device Crashed	**New CVE**	1
	MoveLevel	[T] **Device State**: [Current Level: 0x00/0xfe] MoveLevel: *'move_mode'*: 0x00/0x01 *'move_rate'*: 0x00	Device Crashed	Old CVE	2
7	*On, MoveWithOnOff* Ⓒ	[T] **Device State**: [On Level: 0xfe] MoveWithOnOff: *'move_mode'*: 0x01 *'move_rate'*: 0x00	Device Crashed	**New CVE**	1
	Off, MoveWithOnOff Ⓒ	[T] **Device State**: [On Level: 0x00] MoveWithOnOff: *'move_mode'*: 0x00 *'move_rate'*: 0x00	Device Crashed	**New CVE**	1
	MoveWithOnOff	[T] **Device State**: [On Level: 0x00/0xfe] MoveWithOnOff: *'move_mode'*: 0x00/0x01 *'move_rate'*: 0x00	Device Crashed	Old CVE	2

7.4 Vulnerability Detection Performance (Q3)

Found Vulnerabilities. We conducted fuzzing on a total of 10 devices and found **25 vulnerabilities with 7 CVEs**, including 19 zero-day vulnerabilities (5 new CVEs assigned) and 6 N-day/Fixed vulnerabilities (2 CVEs), as shown in **Table 5**. We perform a study about two cases assigned with new CVEs. Another case is described in **Appendix D**. Then we perform a ablation study and compare each module of ZVDetector with each baseline (B1-B4) in terms of the found vulnerabilities. Finally, we evaluate the efficiency of our combined state fuzzing algorithm.

• **Tuya Smart Bulb (ID=4). (1) [Triggering Method]**: Fuzzer generates a ZCL general message sequence with **dependency** relationship, containing a message ***WriteAttributes*** with the value of field *'device enabled'* is mutated to a negative value, and a normal message ***WriteAttributesResponse***. **[Observations]**: A denial of service (DoS) is triggered but the device still alive, including the command ***ResetFactoryDefault*** of the cluster *Basic*. **(2) [Triggering Method]**: Before sending the message sequence, the message ***RemoveAll*** should be sent to remove all the groups joined by the device, i.e., the **hidden device state** *'group_count'* is set to 0. Fuzzer generates a message sequence of four ZCL general messages with **correlation** relationship. The first two messages belong to the cluster *Groups* and the last two messages belong to the cluster *Scenes*. They are associated through the fields *'group id'* and *'scene id'*. The field *'group_name'* in the message ***AddGroup*** is mutated to contain an excessively long string. Then the field *'group id'* in ***AddScene*** and ***GetSceneMembership*** is aligned with ***AddGroup*** to create a new *'scene id'* and to get the related information of the scene. **[Observations]**: The bulb crashes and requires a reboot to recover. Too long string name may corrupt the group table and lead to device buffer overflow. This correlation helps the fuzzer find several messages that write a specific field first and then query the same field.

• **Sengled Smart Bulb (ID=6).** We discover two vulnerabilities on the device, both triggered by a sequence of two ZCL messages under cluster *LevelControl* with **hidden correlation** relationship: ***Step*** (0x02) and ***Move*** (0x01). **[Triggering Method]**: The **device state** attribute ***current_level*** are set to 0xfe and 0x00. The value of two fields *"step_size"* and *'move_rate'* in these two messages are mutated to 0xff and 0x00. The difference between the two vulnerability triggering methods is that the former reduces the level by ***Step*** and then raises the level by ***Move***, the latter raises the level by ***Step*** and then reduces the level by ***Move***. **[Observations]**: Our analysis indicates that the associated device state may affect the messages that change itself, and invalid move step may cause the device to execute the function incessantly, leading to the DoS.

Ablation Study. [Without Device State]: We first evaluate basic protocol state fuzzing by applying different mutation strategies to individual protocol messages. Both B3 and B4 are able to discover the same two vulnerabilities on ID=2 and ID=5, as shown in **Table 6.** In contrast, we further explore negative values and design an argument serialization method that avoids being sanitized due to OverflowError, and try multiple parameter mutations and MS messages. So we are able to discover six additional vulnerabilities on ID= 2, 3, 5. Next, we evaluate the performance of using only dependency and correlation results for fuzzing. Since the dependency

that triggers vulnerabilities on device ID=4 is relatively simple, both our method and B3 are able to discover the related four cases. Compared to B1, our approach also analyzes hidden correlations and cross-device scenarios, enabling the discovery of one vulnerability on device ID=1 triggered by a ZDP correlated message sequence. **[Combined State]**: Both our approach and B4 identify four vulnerabilities on ID=6 and ID=7 that are triggered by a single mutated message combined with basic state attributes. Most notably, many crashes occur when correlated messages are combined with correlated state attributes, including other four vulnerabilities on ID=6, 7. Finally, by incorporating attribute permissions, we discover the last two vulnerabilities on ID=4, which are triggered by a four-message mutated sequence with a hidden state attribute that could be read and modified by two of these messages.

We analyze that there are **three reasons** why B1-B4 fails to detect other vulnerabilities: **(1)** They ignore many message correlations, including hidden correlations and cross-device correlations, which restricts the kinds of messages that can be used to trigger a vulnerability. **(2)** They fail to collect many MS clusters, attributes and messages, and ignore to discover hidden device states, which neglect many preconditions to trigger vulnerabilities. **(3)** They focus only on protocol state fuzzing and ignore the impact of device state on messages sending and receiving.

Efficiency. Our fuzzing takes 1.8-21.6 hours depending on different vulnerabilities. The most time-consuming case is on device ID=4, triggered by a combined state involving four correlated messages. This is because correlation-based unknown protocol state exploration started later and require mutating various lengths of message sequences combined with much more state attribute settings. Besides, the time overhead for setting device states is relatively low, ranging from 29 minutes to 3.7 hours, since we only update the values of non-overlapping attributes. Finally, two previous state-aware modules may still contain a few false positives introduced by the LLM that not filtered by our validation, which could lead to a slight increase in time cost due to some invalid explorations.

8 Discussion

Manual Effort. Most of the modules in this paper are automated, but some parts require manual effort. Obtaining the Zigbee specification that describes messages **(§4.2)** and development documentation for various device models from different manufacturers **(§5.1)** are all one-time manual efforts. For different devices, the prompts are universal, but the parsing of documentation needs to be adjusted, as the text structure varies between different manufacturers.

Extend to New Protocol. The methodology of combination state fuzzing can be effectively applied to other IoT protocols since they all involve device states and protocol states. The main challenge lies in how to collect the internal device state attributes, as different IoT protocols have significant differences in the definition, data types, and access methods for internal state attributes, with no unified standard. Fortunately, most IoT devices support active querying **(§5.1-Stage 1)**, such as BLE exposes its characteristics through the GATT protocol and allows device state retrieval by sending a Read Characteristic request. In addition, most protocols have defined specification libraries **(§5.1-Stage 2)**, such as the MQTT Paho Client [7] and Mosquitto [6], Android BLE library [4], Finally, other modules (**§4**, **§6**) can be easily migrated to new protocols by slightly changing the prompts and device setup methods. We will apply our method for vulnerability detection on other IoT protocols in future work and update results on our open-source project.

Table 6: The impact of protocol state(PS) and device state(DS) exploration on discovering vulnerabilities. Our results are labeled with (Z) and baselines are labeled with (B1)-(B4).

PS Method / DS Method	Protocol State Awareness		
	Fuzzing of Single Message	+ Single Dependency Based Fuzzing	+ Single Correlation Based Fuzzing
Without Device State	2(B3)/2(B4)/8(Z)	12(B2)/12(Z)	10(B1)/11(Z)
		15(Z) (Combined Two Relationships)	
+ With Device State Awareness (Combined State Fuzzing)			
Attribute Collection	6(B4)/12(Z)	16(Z)	19(Z)
		23(Z) (Combined Two Relationships)	
+ Permission Analysis	12(Z)	16(Z)	21(Z)
		25(Z) (Combined Two Relationships)	

9 Related Work

Research on detecting vulnerabilities in IoT devices has been researched widely and can be broadly categorized into two groups: fuzzing the protocol and analyzing the firmware.

Protocol fuzzing. Most studies are aimed at protocol state fuzzing [21, 23, 25, 28–31, 39, 45] including the handshake phase and cipher exchange phase. They try to break the dependencies of protocol messages to discover vulnerabilities. Joeri et.al [25] and Fiterau et.al [28–30] use protocol state fuzzing to discover vulnerabilities in the implementation of the TLS and DTLS protocols respectively. Other studies [35, 36, 47, 48] collect message formats of APS and ZCL layer and perform format semantic-aware fuzzing.

Firmware Analysis. Most device firmware analysis methods employ static analysis techniques [18, 19, 22, 24, 32, 38, 43]. A few methods [19, 33, 41, 49, 50] utilize emulation techniques. Some methods [20, 27, 40] fuzz through the network. Snipuzz [27] collects message seeds through manufacturers' test APIs, then mutates these seeds based on device feedback. IoTFuzzer [20] and DIANE [40] analyze the companion applications to send the mutated messages that cover critical data flow paths in the application.

10 Conclusion

Inspired by the combination of certain device state and protocol state that can cause device crashes, this paper designs a fuzzing framework called ZVDetector, which can effectively identify such vulnerabilities in Zigbee devices. We propose two state-aware modules, which enables fine-grained profile of device states by discovering more complete internal state attributes, and accurately depict the protocol state by parsing various protocol message formats and identifying various message relationships. A state-guided fuzzer has been designed to efficiently explore the combined space of device states and protocol states, generating test cases comprising mutated protocol message sequences and device state. Experiments demonstrate that ZVDetector achieves high coverage in extracting device endpoint information, high accuracy and richness in message format extraction and relationship discovery, and excellent performance in vulnerability detection.

Ethic Consideration

For all newly discovered vulnerabilities, we have submitted reports to the CVE team. We will publicize these CVE identifiers and firmware version when the vendor releases a patch for these zero-day vulnerabilities, or when they believe that the vulnerability can be publicized without posing the security risk.

Acknowledgment

We thank all the anonymous reviewers for their valuable comments to improve this paper. This work is supported by the National Natural Science Foundation of China (No. 62172251).

References

[1] 2023. CVE-2023-24678: Centralite Pearl Thermostat denial-of-service vulnerability. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-8231.
[2] 2023. CVE-2023-29779: Sengled Dimmer Switch V0.0.9 denial-of-service vulnerability. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-8231.
[3] 2023. CVE-2023-29780: Third Reality Smart Blind 1.00.54 denial-of-service vulnerability. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-8231.
[4] 2024. Android BLE Library. https://github.com/NordicSemiconductor/Android-BLE-Library.
[5] 2024. CNNVD vulnerability database. https://www.cnnvd.org.cn/home/loophole.
[6] 2024. Eclipse Mosquitto Library. https://github.com/eclipse-mosquitto/mosquitto/tree/master.
[7] 2024. Eclipse Paho MQTT Python Client Library. https://github.com/eclipse-paho/paho.mqtt.python/tree/master/src/paho/mqtt.
[8] 2024. IEEE 802.15.4-2003, IEEE Standard for Information technology. https://user.engineering.uiowa.edu/~mcover/lab4/802.15.4-2003.pdf.
[9] 2024. LearnLib: An open framework for automata learning. https://learnlib.de/.
[10] 2024. MITRE CVE vulnerability database. https://cve.mitre.org/cve/search_cve_list.html.
[11] 2024. NVD vulnerability database. https://nvd.nist.gov/vuln/search.
[12] 2024. TLS-Attacker: A Java-based framework for analyzing TLS libraries. https://github.com/tls-attacker/TLS-Attacker.
[13] 2024. Zibgee Cluster Library User Guide. https://www.nxp.com/docs/en/user-guide/JN-UG-3115.pdf.
[14] 2024. Zigbee Market Size Share Analysis- Growth Trends Forecasts(2024 - 2029). https://www.mordorintelligence.com/industry-reports/zigbee-market.
[15] 2024. ZigBee Specification. https://csa-iot.org/wp-content/uploads/2023/04/05-3474-23-csg-zigbee-specification-compressed.pdf.
[16] 2024. Zigbee Traffic Dump Tool: TiWsPc). https://www.ti.com/tool/TIMAC.
[17] 2024. ZigBee Zigpy Library Implementation. https://github.com/zigpy/zigpy/tree/dev.
[18] Zafeer Ahmed, Ibrahim Nadir, Haroon Mahmood, Ali Hammad Akbar, and Ghalib Asadullah Shah. 2020. Identifying mirai-exploitable vulnerabilities in iot firmware through static analysis. In *2020 International Conference on Cyber Warfare and Security (ICCWS)*. IEEE, 1–5.
[19] Daming D Chen, Maverick Woo, David Brumley, and Manuel Egele. 2016. Towards automated dynamic analysis for linux-based embedded firmware.. In *NDSS*, Vol. 1. 1–1.
[20] Jiongyi Chen, Wenrui Diao, Qingchuan Zhao, Chaoshun Zuo, Zhiqiang Lin, XiaoFeng Wang, Wing Cheong Lau, Menghan Sun, Ronghai Yang, and Kehuan Zhang. 2018. IoTFuzzer: Discovering Memory Corruptions in IoT Through App-based Fuzzing.. In *NDSS*.
[21] Yurong Chen, Tian Lan, and Guru Venkataramani. 2019. Exploring effective fuzzing strategies to analyze communication protocols. In *Proceedings of the 3rd ACM Workshop on Forming an Ecosystem Around Software Transformation*. 17–23.
[22] Kai Cheng, Qiang Li, Lei Wang, Qian Chen, Yaowen Zheng, Limin Sun, and Zhenkai Liang. 2018. DTaint: detecting the taint-style vulnerability in embedded device firmware. In *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 430–441.
[23] Lesly-Ann Daniel, Erik Poll, and Joeri de Ruiter. 2018. Inferring OpenVPN state machines using protocol state fuzzing. In *2018 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 11–19.
[24] Drew Davidson, Benjamin Moench, Thomas Ristenpart, and Somesh Jha. 2013. {FIE} on firmware: Finding vulnerabilities in embedded systems using symbolic execution. In *22nd USENIX Security Symposium (USENIX Security 13)*. 463–478.
[25] Joeri De Ruiter and Erik Poll. 2015. Protocol state fuzzing of {TLS} implementations. In *24th USENIX Security Symposium (USENIX Security 15)*. 193–206.
[26] Gianluca Dini and Marco Tiloca. 2010. Considerations on security in zigbee networks. In *2010 IEEE International Conference on Sensor Networks, ubiquitous, and trustworthy computing*. IEEE, 58–65.
[27] Xiaotao Feng, Ruoxi Sun, Xiaogang Zhu, Minhui Xue, Sheng Wen, Dongxi Liu, Surya Nepal, and Yang Xiang. 2021. Snipuzz: Black-box fuzzing of iot firmware via message snippet inference. In *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*. 337–350.
[28] Paul Fiterau-Brostean, Bengt Jonsson, Robert Merget, Joeri De Ruiter, Konstantinos Sagonas, and Juraj Somorovsky. 2020. Analysis of {DTLS} implementations using protocol state fuzzing. In *29th USENIX Security Symposium (USENIX Security 20)*. 2523–2540.
[29] Paul Fiterău-Broştean, Bengt Jonsson, Konstantinos Sagonas, and Fredrik Tåquist. 2022. Dtls-fuzzer: A dtls protocol state fuzzer. In *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 456–458.
[30] Paul Fiterau-Brostean, Bengt Jonsson, Konstantinos Sagonas, and Fredrik Tåquist. 2023. Automata-Based Automated Detection of State Machine Bugs in Protocol Implementations.. In *NDSS*.
[31] Hugo Gascon, Christian Wressnegger, Fabian Yamaguchi, Daniel Arp, and Konrad Rieck. 2015. Pulsar: Stateful black-box fuzzing of proprietary network protocols. In *Security and Privacy in Communication Networks: 11th EAI International Conference, SecureComm 2015, Dallas, TX, USA, October 26-29, 2015, Proceedings 11*. Springer, 330–347.
[32] Grant Hernandez, Farhaan Fowze, Dave Tian, Tuba Yavuz, and Kevin RB Butler. 2017. Firmusb: Vetting usb device firmware using domain informed symbolic execution. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2245–2262.
[33] Mingeun Kim, Dongkwan Kim, Eunsoo Kim, Suryeon Kim, Yeongjin Jang, and Yongdae Kim. 2020. Firmae: Towards large-scale emulation of iot firmware for dynamic analysis. In *Proceedings of the 36th Annual Computer Security Applications Conference*. 733–745.
[34] Kaizheng Liu, Ming Yang, Zhen Ling, Yue Zhang, Chongqing Lei, Junzhou Luo, and Xinwen Fu. 2024. Riotfuzzer: Companion app assisted remote fuzzing for detecting vulnerabilities in iot devices. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2341–2354.
[35] Xiaoyue Ma, Qiang Zeng, Haotian Chi, and Lannan Luo. 2023. No more companion apps hacking but one dongle: Hub-based blackbox fuzzing of iot firmware. In *Proceedings of the 21st Annual International Conference on Mobile Systems, Applications and Services*. 205–218.
[36] Ruijie Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. 2024. Large language model guided protocol fuzzing. In *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)*.
[37] Philipp Morgner, Stephan Mattejat, Zinaida Benenson, Christian Müller, and Frederik Armknecht. 2017. Insecure to the touch: Attacking ZigBee 3.0 via touchlink commissioning. In *Proceedings of the 10th ACM conference on security and privacy in wireless and mobile networks*. 230–240.
[38] Ibrahim Nadir, Zafeer Ahmad, Haroon Mahmood, Ghalib Asadullah Shah, Farrukh Shahzad, Muhammad Umair, Hassam Khan, and Usman Gulzar. 2019. An auditing framework for vulnerability analysis of IoT system. In *2019 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 39–47.
[39] Roberto Natella. 2022. Stateafl: Greybox fuzzing for stateful network servers. *Empirical Software Engineering* 27, 7 (2022), 191.
[40] Nilo Redini, Andrea Continella, Dipanjan Das, Giulio De Pasquale, Noah Spahn, Aravind Machiry, Antonio Bianchi, Christopher Kruegel, and Giovanni Vigna. 2021. Diane: Identifying fuzzing triggers in apps to generate under-constrained inputs for iot devices. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 484–500.
[41] Mengfei Ren, Xiaolei Ren, Huadong Feng, Jiang Ming, and Yu Lei. 2021. Z-Fuzzer: device-agnostic fuzzing of Zigbee protocol implementation. In *Proceedings of the 14th ACM Conference on Security and Privacy in Wireless and Mobile Networks*. 347–358.
[42] Eyal Ronen, Adi Shamir, Achi-Or Weingarten, and Colin O'Flynn. 2017. IoT goes nuclear: Creating a ZigBee chain reaction. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 195–212.
[43] Vinay Sachidananda, Suhas Bhairav, and Yuval Elovici. 2020. OVER: Overhauling vulnerability detection for IoT through an adaptable and automated static analysis framework. In *Proceedings of the 35th Annual ACM Symposium on Applied Computing*. 729–738.
[44] Narmeen Shafqat, Daniel J Dubois, David Choffnes, Aaron Schulman, Dinesh Bharadia, and Aanjhan Ranganathan. 2022. Zleaks: Passive inference attacks on Zigbee based smart homes. In *International Conference on Applied Cryptography and Network Security*. Springer, 105–125.
[45] Congxi Song, Bo Yu, Xu Zhou, and Qiang Yang. 2019. SPFuzz: a hierarchical scheduling framework for stateful network protocol fuzzing. *IEEE Access* 7 (2019), 18490–18499.
[46] Grace Hanusha Talakala and Jyotsna Bapat. 2021. Detecting spoofing attacks in Zigbee using device fingerprinting. In *2021 IEEE 18th Annual Consumer Communications & Networking Conference (CCNC)*. IEEE, 1–2.
[47] Jincheng Wang, Le Yu, and Xiapu Luo. 2024. Llmif: Augmented large language model for fuzzing iot devices. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 196–196.
[48] Xian Wang and Shuang Hao. 2022. Don't Kick Over the Beehive: Attacks and Security Analysis on Zigbee. In *Proceedings of the 2022 ACM SIGSAC Conference*

on Computer and Communications Security. 2857–2870.
[49] Yaowen Zheng, Ali Davanian, Heng Yin, Chengyu Song, Hongsong Zhu, and Limin Sun. 2019. {FIRM-AFL}:{High-Throughput} greybox fuzzing of {IoT} firmware via augmented process emulation. In *28th USENIX Security Symposium (USENIX Security 19)*. 1099–1114.
[50] Wei Zhou, Lan Zhang, Le Guan, Peng Liu, and Yuqing Zhang. 2022. What your firmware tells you is not how you should emulate it: A specification-guided approach for firmware emulation. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 3269–3283.
[51] Tobias Zillner and Sebastian Strobl. 2015. ZigBee exploited: The good, the bad and the ugly. *Black Hat–2015. Available online: https://www. blackhat. com/docs/us-15/materials/us-15-Zillner-ZigBee-Exploited-The-Good-The-Bad-And-The-Ugly. pdf (accessed on 21 March 2018)* (2015).

A Details of Zigpy Library Analysis

```
# Zigpy ZHA Library-general.py
class Groups(Cluster):
    class ServerCommandDefs(BaseCommandDefs):
        add: Final = ZCLCommandDef(
            id=0x00,
            schema={
                "group_id": t.Group,
                "group_name": t.LimitedCharString(16)},
            direction=False,)
```

```
# Zigpy ZHA Library-general.py
class Groups(Cluster):
    class ClientCommandDefs(BaseCommandDefs):
        add_response: Final = ZCLCommandDef(
            id=0x00,
            schema={"status": foundation.Status,
                    "group_id": t.Group},
            direction=True,
        )
```

Figure 8: An example of the Zigpy code that define received commands and generated commands supported by the cluster *Groups*.

```
# Zigpy ZHA Library - general.py
class LevelControl(Cluster):
    class AttributeDefs(BaseAttributeDefs):
        current_level: Final = ZCLAttributeDef(
            id=0x0000, type=t.uint8_t,
            access="rps", mandatory=True
        )
        on_level: Final = ZCLAttributeDef(
            id=0x0011, type=t.uint8_t, access="rw")
```

```
# HomeAssistant ZHA Library - light.py
class BaseLight(LogMixin, light.LightEntity):
    _FORCE_ON = False
    _DEFAULT_MIN_TRANSITION_TIME = 0
    def __init__(self, *args, **kwargs):
        self._zha_device: ZHADevice = None
        super().__init__(*args, **kwargs)
        self._attr_min_mireds: int | None = 153
        self._attr_max_mireds: int | None = 500
```

Figure 9: The code that define cluster state attributes in two python files of the Zigpy library and Home Assistant ZHA.

B Details of the Manufacturer Development Document Analysis and Related Results

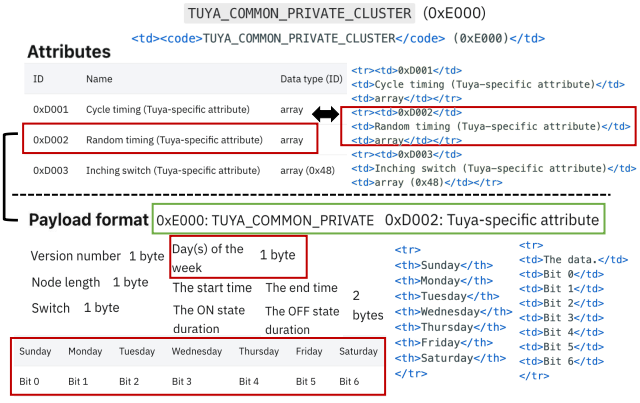


Figure 10: An example of extracting the attributes of the Manufacturer-specific cluster *Tuya_Common_Private_Cluster* from the Tuya development document in HTML file.

Various manufacturers have their customized clusters, attributes and corresponding commands to achieve specific functions, such as Tuya Smart Plug with power-off memory and cycle timing. In **§5.1.2**, we analyze the development documentation publicly released by each manufacturer. An example of the development documentation for a Tuya device is shown in **Figure 10**. We parse the tabular(<th></th>, <td></td>) from the HTML and extract the content that contains cluster information.

Table 7: Number of Manufacturer-specific(MS) attr ibutes (including hidden attributes) and MS ZCL messages found by ZVDetector on 8 devices in the testbed.

Number / Kind / Device	Attributes	Messages
Tuya Multi-gang Switch(ID=2)	7	14
Tuya Smart Plug(ID=3)	12	18
Tuya Smart Bulb(ID=4)	18	33
Sengled Dimmer Switch(ID=5)	1	2
Sengled Smart Bulb(ID=6)	2	3
Tuya OMEG Smart Valve(ID=8)	1	8
Third Reality Bulb(ID=9)	2	3
Third Reality Plug(ID=10)	1	2
Total	44	83

In total, we find 44 manufacturer-specific (MS) attributes and 83 MS messages on 9 clusters of 8 devices, as shown in **Table 7**. Since the vendors Sengled and Third Reality do not have relevant open source documentation, we analyze the **ZHA Device Handler** library used to represent device models in the ZHA component of Home Assistant. Even so, there are still several device models that are not included in this library. To solve this problem, we apply active probing to collect MS attributes and messages. In §4.1.1, we have collected all clusters marked as "Unknown", which do not belong to the standard clusters. So we can guess that they may be manufacturer-specific clusters. For these clusters, we send messages ***Read_Attributes*** with *'attribute_id'* from 0x00 to 0xFE to search for all possible attributes. Similarly, we send messages with command ID from 0x0000 to 0xFFFE to search for all possible messages.

These MS attributes and messages do not all belong to MS clusters. **Table 8** lists the extraction results of the four Tuya devices used in the testbed. There are also many manufacturer-defined attributes and messages on the general clusters *On/Off* and *LevelControl*. In addition, we find many hidden MS attributes by analyzing MS messages. For example, four hidden MS attributes are found on the cluster *ColorControl* of Tuya Smart Bulb, namely 'Color', 'Color Temperature', 'White Gradient' and 'Color Graident'.

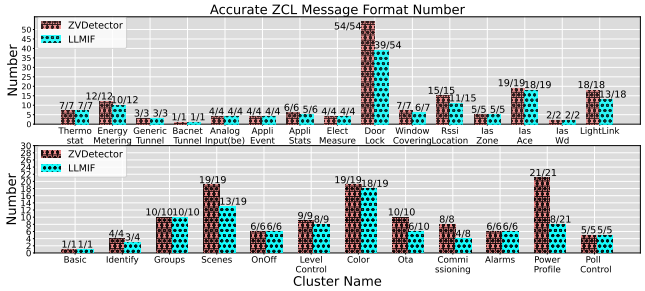


Figure 11: Number of correct message formats extracted by ZVDetector and LLMIF for 27 common-used clusters.

Table 8: Manufacturer-specific clusters, attributes and commands of Tuya vendor collected by ZVDetector. [H] means hidden attributes, [W] means message *Write_Attributes* and [P] means message *Report_Attributes*.

Device \ Kind	Cluster	Cluster ID	Attribute(Attribute ID)	Command
Tuya Smart Switch (ID=2)	On/Off	0x0006	Backlight Switch(0x5000)	[W] + [P]
			Indicator Status(0x8001)	[W] + [P]
			Restart Status(0x8002)	[W] + [P]
	Tuya Common Private	0xE000	Random Timing(0xD001)	0xF7 + [P]
			Cycle Timing(0xD002)	0xF8 + [P]
			Inching Switch(0xD003)	0xFB + [P]
	Tuya Electrician Private	0xE001	Restart Status(0xD010)	[W] + [P]
Tuya Smart Plug (ID=3)	N/A	0x1888	N/A	N/A
	On/Off	0x0006	Child Lock(0x8000)	[W] + [P]
			Light Mode(0x8001)	[W] + [P]
			Relay Status(0x8002)	[W] + [P]
	Tuya Common Private	0xE000	Cycle Timing(0xD001)	0xF8 + [P]
			Random Timing(0xD002)	0xF7 + [P]
			Inching Switch(0xD003)	0xFD + [P]
	Tuya Electrician Private	0xE001	Calibration Result(0xD000)	N/A
			Fault(0xD001)	[P]
			Voltage Coefficient(0xD002)	[P]
			Electric Coefficient(0xD003)	[P]
			Current Coefficient(0xD004)	[P]
			Power Coefficient(0xD005)	[P]
			N/A	Historical Power(0xE1)
Tuya Smart Bulb (ID=4)	Color Control	0x0300	Color Temperature(0xE000)[H]	0xE0 + [P]
			Color(0xE100)[H]	0xE1 + [P]
			Mode(0xF000)	0xF0 + [P]
			Color Intensity(0xF001)	N/A
			Global Data(0xF002)	N/A
			Scene Data(0xF003)	0xF1 + [P]
			N/A	Adjust Music(0xF2)
			Sleep(0xF007)	0xF4 + [P]
			Awake(0xF008)	0xF5 + [P]
			Biorhythm Lighting(0xF009)	0xF6 + [P]
			Random Timing(0xF00A)	0xF7 + [P]
			Cycle Timing(0xF00B)	0xF8 + [P]
			Power-on Behavior(0xF00C)	0xF9 + [P]
			Do Not Disturb(0xF00D)	0xFA + [P]
			On/Off Gradient(0xF00E)	0xFB + [P]
			White Gradient(0xF013)[H]	0xFD + [P]
			Color Gradient(0xF014)[H]	0xFE + [P]
	Level Control	0x0008	Brightness(0xF000)[H]	0xF0 + [P]
	On/Off	0x0006	CountDown(0xF000)	0xF0 + [P]
Tuya Smart Valve (ID=8)	Tuya Electrician Private	0xE001	Raw Data(0xD040)	[P] Fault Report[P] Work State[P] Once Using Time[P] Scheduled Irrigation(0xFC) Cycle Irrigation(0xFD) Weather Delay(0xFE) Irrigation Time(0xFE)

C Comparison of ZVDetector and LLMIF in Message Format Extraction

LLMIF (**B1**) is able to correctly extract 220 non-MS ZCL cluster-specific messages, but the accuracy is lower on some clusters like *DoorLock* and *PowerProfile*. Our analysis reveals that some message descriptions for these clusters are vague or missing in the Zigbee Alliance specification. Besides, in the cluster *Color*, although LLMIF identifies most message formats correctly, it fails to distinguish the semantics of the field *'transition time'* and the field *'duration'* in the message ***StepColor***. **Figure 11** shows the number of messages correctly extracted in each common-used cluster by ZVDetector and LLMIF. We also correctly extract 17 ZCL general messages from Zigpy library and utilize LLM to successfully generate message formats of other layers.

D Details of Vulnerabilities Found in Testbed

Figure 12 shows all the devices used in the experimental environment, including two Zigbee gateways (coordinators), a CC2531 usb dongle for capturing Zigbee traffic, and 10 devices in the testbed. In addition, we also test other 2 Zigbee devices, Aqara Door/Window Sensor and Honeywell Smoke Sensor, and will subsequently update the test results and related data in our open source project. Sensor devices from many vendors are designed to be low-power types that remain silent during long-term communication. This state means that it will periodically report sensed data such as light intensity, temperature and humidity, but not respond to messages sent by the coordinator. Therefore, fuzzing for the sensor is limited to a very small time window, only 10 to 40 seconds after the commissioning phase is completed. This makes it necessary to keep sending the NWK command *Leave* (cid: 0x04) to restart the sensor commissioning process. **[Future Work]**: Since many protocol devices also have device states and protocol states, we are working to extend ZVDetector to more protocols, such as a Z-Wave device Fibaro Smart Button shown in the **Figure 12**.

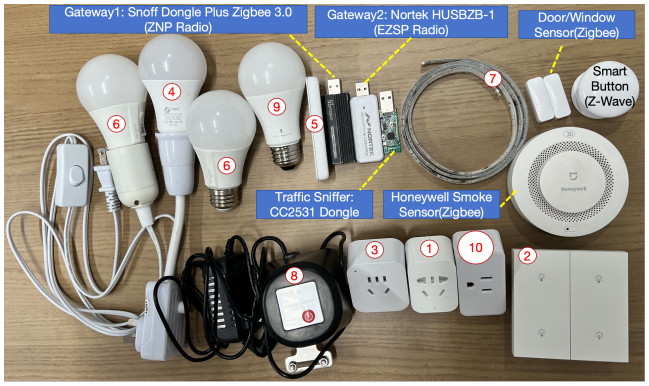


Figure 12: Snapshot of the devices used for the experiment.

We have already conducted case studies on a zero-day vulnerabilities found by ZVDetector in **§7.4**. Here we analyze another vulnerabilities caused by unexpected combined state.

• **Sengled Light Strip (ID=7). [Zero-day Vulnerabilities]**: Two zero-day vulnerabilities on this device are related to the product space of device state and protocol state. **[Triggering Method]**: The **device state** attribute *'on_level'* needs to be set to 0xfe and 0x00 respectively before sending the mutation messages. Specifically, the **hidden correlation** relationship between the messages ***On***(0x01)/***Off***(0x00) and ***MoveWithOnOff***(0x05) is obtained by LLM prompt (Figure 4), since there is no common field in their payloads, but changing the device attribute *'on_off'* at the same time. The two vulnerabilities differ in their triggering methods: the first method turns on the device through message ***On***, then lowers the light intensity and turns off the device through message ***MoveWithOnOff***. The second method turns off the device through message ***Off***, then turns on the device and increases the light intensity through message ***MoveWithOnOff***. **[Observations]**: Both of these vulnerabilities have a similar cause as the vulnerability found in the Sengled Smart Bulb (ID=6). **[N-day/fixed Vulnerabilities]**: The message ***MoveWithOnOff*** (cid:0x05) is a server command of cluster *LevelControl*, which can change the light intensity and turn on/off the device. Specifically, this message has two fields: *'move_mode'* and *'move_rate'*. When the field *'move_mode'* is set to 0x00, this message can turn on the device and adjusts the device from minimum to maximum brightness according to *'move_rate'* within a certain period of time. When *'move_mode'* is set to 0x01, the process is reversed. **[Triggering Method]**: When **device state** attribute *'on_level'* is set to 0x00 or 0xfe, this message with the field *'move_mode'* set to 0x00 or 0x01 and the field *'move_rate'* set to 0x00 can cause a Dos to the device. **[Observations]**: The device needs to be restored to factory settings to receive other commands. Invalid value of the field *'move_rate'* can cause the device to be unable to process the message and corrupt the firmware.